

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

连通性问题

2. 实验目的

关系 R 具有对称性和传递性。数对 $p\ q$ 表示 pRq , p 和 q 是 0 或自然数, p 不等于 q 。

要求写一个程序将数对序列进行过滤, 如果一个数对可以通过前面数对的传递性得到, 则将其滤去。例如:

输入	输出	连通性
3 4	3 4	
4 9	4 9	
8 0	8 0	
2 3	2 3	
5 6	5 6	
2 9		2-3-4-9
5 9	5 9	
7 3	7 3	
4 8	4 8	
5 6		5-6
0 2		0-8-4-3-2
6 1	6 1	

其中数对 2 9 和 0 2 可由之前数对的连通关系得到, 故不做输出。

3. 算法设计

- 1.并查集算法设计：
 - ①创建数组，数组的大小为元素的自然数范围的最大值
 - ②初始化：数组值初始化为下标，保证该元素指向自身，即没有有序对
 - ③合并：合并 i 和 j，找到 i 和 j 的祖先，进行合并
 - ④查找祖先：如果祖先为自己（即没有有序对情况）返回自身；否则，进行路径压缩，所有的结点共用同一个祖先
- 2.主函数里的逻辑：
 - ①输入
 - ②初始化，元素上限设为 10000，否则会崩溃
 - ③对每一组元素，先查、后并
 - ④输出，如果查不到公共祖先则输出

4. 程序运行与测试

1. 并查集的数组定义

```
#define NUM 100000  
int fa[NUM];//定义
```

- 2.并查集的数组初始化

```
void init(int n)//初始化  
{  
    for (int i = 0; i <= n; i++)  
        fa[i] = i;//初始化时祖先为自己  
}
```

- 3.并查集的查找祖先函数

```

int UFS_find(int x)//查找x的祖先
{
    if (fa[x] == x)
        return x;//祖先为自己
    else
    {
        fa[x] = UFS_find(fa[x]); //路径压缩
        return fa[x];
    }
}

```

4.并查集的合并函数

```

void UFS_union(int i, int j)//合开: i指向j
{
    int i_fa = UFS_find(i); //找到i的祖先
    int j_fa = UFS_find(j); //找到j的祖先
    fa[i_fa] = j_fa;
}

```

5.主函数的初始化、输入输出、先查后并逻辑

```

int main()
{
    init(10000);
    int a,b;
    while(cin>>a>>b)
    { //先查 后并
        if(UFS_find(a)!=UFS_find(b))
            cout<<a<<" "<<b<<endl;
        UFS_union(a,b);
    }
    return 0;
}

```

5. 实验总结与心得

- 1.题目的逻辑比较简单
- 2.重点在于掌握并查集的基础模板，定义数组、初始化、合并、查找祖先，其中查找祖先的函数里，为了提高代码的时间效率，需要使用的压缩路径的方法
- 3.同时要考虑题目中的自然数的范围，但是又不能超时

第二题

1. 实验题目

家族查询

2. 实验目的

某个家族人员过于庞大，要判断两个是否是亲戚，确实还很难，现在给出某个亲戚关系图，求任意给出的两个人是否具有亲戚关系。

规定：x 和 y 是亲戚，y 和 z 是亲戚，那么 x 和 z 也是亲戚。如果 x,y 是亲戚，那么 x 的亲戚都是 y 的亲戚，y 的亲戚也都是 x 的亲戚。（人数 ≤ 5000 ，询问亲戚关系次数 ≤ 5000 ）。

3. 算法设计

- 1.并查集算法设计：
 - ①创建数组，数组的大小为元素的自然数范围的最大值
 - ②初始化：数组值初始化为下标，保证该元素指向自身，即没有有序对
 - ③ 合并：合并 i 和 j，找到 i 和 j 的祖先，进行合并
 - ④查找祖先：如果祖先为自己（即没有有序对情况）返回自身；否则，进行路径压缩，所有的结点共用同一个祖先
2. 主函数里的逻辑：
 - ①输入，数组组数 T 第一层循环 T 次；
 - ②人数 n 初始化 n；
 - ③亲戚关系个数 m 第二层第一个循环 m 次；
 - ④在这一个循环里，输入 u v 的值，对 u v 进行合并；
 - ⑤询问次数 q 第二层第二个循环 q 次；
 - ⑥在这一个循环里，输入 a b 的值，对 a b 进行查找祖先比较；
 - ⑦查找比较有共同祖先则输出 Yes，否则 No

4. 程序运行与测试

1. 并查集的数组定义

```
#define NUM 100000  
int fa[NUM]; //定义
```

2. 并查集的数组初始化

```
void init(int n) //初始化  
{  
    for (int i = 0; i <= n; i++)  
        fa[i] = i; //初始化时祖先为自己  
}
```

3. 并查集的查找祖先函数

```
int UFS_find(int x) //查找x的祖先  
{  
    if (fa[x] == x)  
        return x; //祖先为自己  
    else  
    {  
        fa[x] = UFS_find(fa[x]); //路径压缩  
        return fa[x];  
    }  
}
```

4. 并查集的合并函数

```
void UFS_union(int i, int j) //合并: i指向j  
{  
    int i_fa = UFS_find(i); //找到i的祖先  
    int j_fa = UFS_find(j); //找到j的祖先  
    fa[i_fa] = j_fa;  
}
```

5. 主函数的输入输出、循环的逻辑、初始化、合并、查找

```

int T;//数据组数
cin>>T;
while(T-->0)
{
    int n;//n个人
    int m;//存在m个亲戚关系
    cin>>n>>m;

    init(n);//初始化

    while(m-->0)
    {
        int u,v;//有亲戚关系 m行
        cin>>u>>v;
        UFS_union(u,v);
    }

    int q;//询问
    cin>>q;

    while(q-->0)
    {
        int a,b;//询问是否有亲戚关系 q行
        cin>>a>>b;
        if(UFS_find(a)==UFS_find(b))
            cout<<"Yes"<<endl;
        else
            cout<<"No"<<endl;
    }
    cout<<endl;
}

```

5.实验总结与心得

- 1.实验和上一题相比较是递进的关系
- 2.还是熟悉并查集的模板
- 3.提升的地方在对题目的理解，要考虑如何设计循环，如何在循环里何时输入输出，何时使用初始化、使用合并、使用查找等等

第三题

1. 实验题目

多叉树与二叉树

2. 实验目的

给出一棵有 n 个点的以 1 为根节点的有根多叉树，请把它转成左儿子右兄弟的二叉树形式，并输出其层次遍历顺序。

注意，每个节点的左儿子一定使用它的所有儿子中的编号最小的，右兄弟一定使用比它编号大的兄弟中最小的兄弟的编号。层次遍历需严格按照自上而下，自左而右访问树的节点。

3. 算法设计

1. 根据输入提供的信息，使用邻接表表示多叉树，建立结点之间的关系
2. 转化为左儿子右兄弟的表示：
① 左儿子是当前所有结点中编号最小的一个
② 有兄弟是同层次里比他大的所有节点里最小的一个（因此后序需要考虑排序问题）
3. 输出：层序遍历输出，可以使用队列来实现，进行遍历

4. 程序运行与测试

1. 邻接表和数组表示

```
vector<int> children[MAXN]; // 多叉树的邻接表
int leftChild[MAXN];        // 左儿子
int rightSibling[MAXN];      // 右兄弟
```

2. 遍历结点构建左儿子右兄弟的结点关系

```
void convertToBinaryTree(int n) {
    // 遍历每个节点，给出其左儿子和右兄弟
    for (int i = 1; i <= n; i++) {
        if (!children[i].empty()) {
            sort(children[i].begin(), children[i].end()); // 按编号排序子节点
            leftChild[i] = children[i][0]; // 左儿子是最小编号子节点
            for (size_t j = 0; j < children[i].size() - 1; j++) {
                rightSibling[children[i][j]] = children[i][j + 1]; // 构建右兄弟
            }
        }
    }
}
```

3. 层序遍历，其中利用了队列

```

void levelOrderTraversal(int root) {
    queue<int> q;
    q.push(root);
    vector<int> result;

    while (!q.empty()) {
        int current = q.front();
        q.pop();
        result.push_back(current);

        if (leftChild[current]) q.push(leftChild[current]);    // 左儿子入队
        if (rightSibling[current]) q.push(rightSibling[current]); // 右兄弟入队
    }

    for (size_t i = 0; i < result.size(); i++) {
        if (i > 0) cout << " ";
        cout << result[i];
    }
    cout << endl;
}

```

5. 实验总结与心得

- 1.实验题难度适中，需要根据多叉树与二叉树的转换逻辑，对函数模块进行设计
- 2.同时要注意到同层次的结点之间也有大小的顺序，需要使用 sort，每个节点的左儿子一定使用它的所有儿子中的编号最小的，右兄弟一定使用比它编号大的兄弟中最小的兄弟的编号。层次遍历需严格按照自上而下，自左而右访问树的节点
- 3.层序遍历的知识之前已经学习过了，重点是使用队列进行操作